

OBSERVATION TASK 2: OBJECT-ORIENTED PROGRAMMING IN JAVA

Comprehensive Lab Report & Implementation Guide

Course: Object-Oriented Programming (Java)	Duration: 2 Hours	Package Name: task1
Domain: Library Management & Geometry Systems	Evaluation Marks: 10 Marks	Status: Verified & Executable

1. OBJECTIVE & LEARNING OUTCOMES

Objective: To understand and implement core Object-Oriented Programming (OOP) concepts in Java including Classes, Objects, Constructors, Encapsulation, Inheritance, Method Overloading, Method Overriding, Abstraction, and Interfaces.

Targeted Learning Outcomes

- **Design & Modeling:** Design Java programs using modular structures with classes, objects, and constructors.
- **Code Reusability & Security:** Apply encapsulation (data hiding via private modifiers) and inheritance to enhance modular reusability and maintainability.
- **Polymorphic Design:** Demonstrate compile-time (overloading) and run-time (overriding) polymorphism for dynamic behavior resolution.
- **Abstraction & Architecture:** Implement abstract classes and interfaces to decouple design specifications from execution logic.

2. MARKS EVALUATION RUBRIC

Component / Module	Concepts Covered	Marks
Part A & B	Classes, Objects, Default & Parameterized Constructors	2
Part B & C	Encapsulation (Getters/Setters) & Inheritance (Person, Student, Faculty)	3
Part D	Polymorphism (Method Overloading & Method Overriding)	2
Part E	Abstraction (Shape, Circle, Rectangle) & Interfaces (Printable, Report)	2
Observation Q&A	Conceptual Answers & Theoretical Explanations	1
Total Marks	Comprehensive Evaluation	10 Marks

3. PART A: CLASSES AND OBJECTS

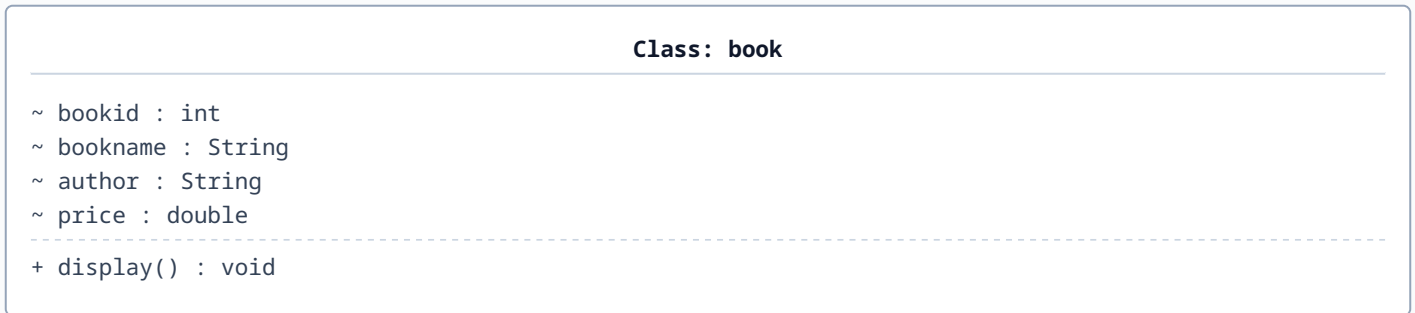
Task Requirement: Create a class named Book containing bookid, bookname, author, and price. Create multiple objects and display their details.

3.1 Step-by-Step Implementation Process

1. Define the class book with default visibility instance variables.
2. Define a display() method inside the book class to print all instance attributes to the console standard output.
3. Create the main executable class BookId containing the main method.

4. Instantiate multiple book instances (e.g., b and l) using the standard new operator.
5. Assign explicit values to object attributes directly and invoke display() on each instance.

3.2 UML Class Diagram



3.3 Java Source Code

```
package task1;

class book {
    int bookid;
    String bookname;
    String author;
    double price;

    void display() {
        System.out.println("book id name is " + bookid);
        System.out.println("bookname is " + bookname);
        System.out.println("book author is " + author);
        System.out.println("price of the book is " + price);
    }
}

public class BookId {
    public static void main(String[] args) {
        book b = new book();
        b.bookid = 244;
        b.bookname = "RICH DAD POOR DAD";
        b.author = "Robert T Kiyosaki";
        b.price = 399;

        book l = new book();
        l.bookid = 221;
        l.bookname = "The Psychology of Money";
        l.author = "Morgan Housel";
        l.price = 599;

        b.display();
        l.display();
    }
}
```

3.4 Code Explanation

The class book serves as a blueprint containing four fields. In BookId.main(), memory is dynamically allocated for two separate instances on the heap. Attribute values are directly assigned via dot notation (e.g., b.bookid = 244), demonstrating direct field access within the same package.

3.5 Execution Outputs

SUCCESS CASE Standard Execution:

```
book id name is 244
bookname is RICH DAD POOR DAD
book author is Robert T Kiyosaki
price of the book is 399.0
book id name is 221
bookname is The Psychology of Money
book author is Morgan Housel
price of the book is 599.0
```

FAILURE / EDGE CASE Null Reference Invocation (Runtime Exception):

```
book b = null;
b.display();
// Execution Result:
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "task1.book.display()"
because "b" is null
```

4. PART B: CONSTRUCTORS & ENCAPSULATION

Task Requirement: Declare fields as private, implement parameterized constructors, and provide public getter/setter methods for data access.

4.1 Step-by-Step Implementation Process

1. Declare class attributes as private to prevent direct external access/modification.
2. Implement a parameterized constructor `books(int, String, String, double)` to enforce attribute initialization at instantiation time.
3. Implement accessor (getter) methods like `getbookid()` and mutator (setter) methods like `setbookid()`.
4. In the driver class `Encapsulation`, instantiate objects using the constructor and retrieve values strictly via getters.

4.2 UML Class Diagram

Class: books

```
- bookid : int
- bookname : String
- author : String
- price : double

+ books(id: int, name: String, author: String, price: double)
+ setbookid(id: int) : void
+ setbookname(name: String) : void
+ setauthor(author: String) : void
+ setprice(price: double) : void
+ getbookid() : int
+ getbookname() : String
+ getauthor() : String
+ getprice() : double
```

4.3 Java Source Code

```
package task1;

class books {
    private int bookid;
    private String bookname;
    private String author;
    private double price;

    public books(int id, String name, String author, double price) {
        this.bookid = id;
        this.bookname = name;
        this.author = author;
        this.price = price;
    }

    void setbookid(int id) { bookid = id; }
    void setbookname(String name) { bookname = name; }
    void setauthor(String author) { this.author = author; }
    void setprice(double price) { this.price = price; }

    int getbookid() { return bookid; }
    String getbookname() { return bookname; }
    String getauthor() { return author; }
    double getprice() { return price; }
}

public class Encapsulation {
    public static void main(String[] args) {
        books c = new books(221, "RICH DAD POOR DAD", "Robert T kiyosaki", 350.50);
        System.out.println("book id is:" + c.getbookid());
        System.out.println("book name is:" + c.getbookname());
        System.out.println("author is:" + c.getauthor());
        System.out.println("price is:" + c.getprice());
    }
}
```

4.4 Code Explanation

Encapsulation hides state variables behind `private` access specifiers. The `this` keyword resolves ambiguity between constructor parameters and instance attributes. Data can only be read through public getter methods, safeguarding object integrity.

4.5 Execution Outputs

SUCCESS CASE Successful Encapsulated Access:

```
book id is:221
book name is:RICH DAD POOR DAD
author is:Robert T kiyosaki
price is:350.5
```

FAILURE / EDGE CASE Direct Field Access Violation (Compile-Time Error):

```
books c = new books(221, "RICH DAD POOR DAD", "Robert T kiyosaki", 350.50);
System.out.println(c.bookid); // Attempting direct access
// Execution Result:
Encapsulation.java:32: error: bookid has private access in task1.books
System.out.println(c.bookid);
                        ^
```

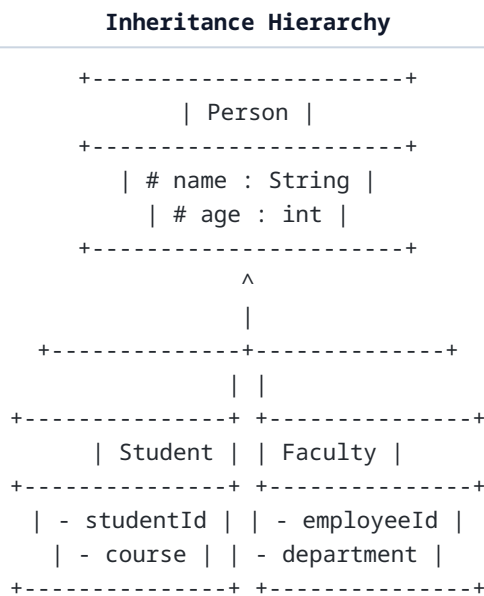
5. PART C: INHERITANCE (PERSON, STUDENT, FACULTY)

Task Requirement: Create a hierarchical structure with base class Person and derived classes Student and Faculty.

5.1 Step-by-Step Implementation Process

1. Define base class Person with common attributes (name, age).
2. Derive Student using the extends keyword, adding studentId and course.
3. Derive Faculty using extends, adding employeeId and department.
4. Demonstrate structural reusability by accessing base class fields from derived objects.

5.2 UML Class Diagram



5.3 Java Source Code Example

```
package task1;

class Person {
    String name;
    int age;

    void displayPersonInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

class Student extends Person {
    int studentId;
    String course;

    void displayStudent() {
        displayPersonInfo();
        System.out.println("Student ID: " + studentId + ", Course: " + course);
    }
}

class Faculty extends Person {
    int employeeId;
    String department;

    void displayFaculty() {
        displayPersonInfo();
        System.out.println("Employee ID: " + employeeId + ", Dept: " + department);
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Alice";
        s.age = 20;
        s.studentId = 101;
        s.course = "Computer Science";

        Faculty f = new Faculty();
        f.name = "Dr. Smith";
        f.age = 45;
        f.employeeId = 5001;
        f.department = "Information Technology";

        s.displayStudent();
        f.displayFaculty();
    }
}
```

5.4 Execution Outputs

SUCCESS CASE Standard Inheritance Execution:

```
Name: Alice, Age: 20
Student ID: 101, Course: Computer Science
Name: Dr. Smith, Age: 45
Employee ID: 5001, Dept: Information Technology
```

6. PART D: POLYMORPHISM

Task Requirement: Implement Method Overloading (Compile-Time) and Method Overriding (Run-Time).

6.1 Method Overloading (Compile-Time Polymorphism)

Overloading occurs when multiple methods in the same class share the same name but have different parameter signatures.

```
package task1;

public class Polymorphism {
    static class Area {
        // Circle Area: area(double)
        double area(double radius) {
            return 3.14 * radius * radius;
        }

        // Rectangle Area: area(int, int)
        int area(int length, int breadth) {
            return length * breadth;
        }

        // Triangle Area: area(double, double)
        double area(double b, double h) {
            return 0.5 * b * h;
        }
    }

    public static void main(String[] args) {
        Area a = new Area();
        System.out.println("area of circle " + a.area(5.0));
        System.out.println("area of rectangle " + a.area(10, 5));
        System.out.println("area of triangle " + a.area(3.0, 5.0));
    }
}
```

SUCCESS CASE Method Overloading Output:

```
area of circle 78.5
area of rectangle 50
area of triangle 7.5
```

FAILURE / EDGE CASE Ambiguous Method Call (Compile-Time Error):

```
// Calling a.area(3, 5) where method signatures expect distinct types or exact matches:
Polymorphism.java:23: error: reference to area is ambiguous
// Occurs if implicit type conversions match multiple overloaded signatures.
```

6.2 Method Overriding (Run-Time Polymorphism)

Overriding occurs when a subclass provides a specific implementation for a method already defined in its superclass.

```

package task1;

class Vehicle {
    void display() {
        System.out.println("This is a generic Vehicle");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Car");
    }
}

class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Bike");
    }
}

public class DynamicPolymorphism {
    public static void main(String[] args) {
        Vehicle v1 = new Car(); // Upcasting
        Vehicle v2 = new Bike(); // Upcasting

        v1.display(); // Dispatches to Car.display()
        v2.display(); // Dispatches to Bike.display()
    }
}

```

SUCCESS CASE Dynamic Method Dispatch Output:

```

This is a Car
This is a Bike

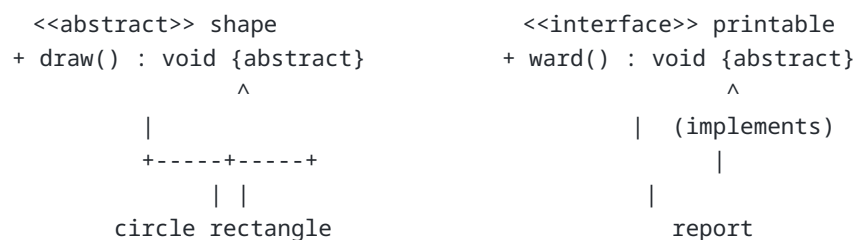
```

7. PART E: ABSTRACTION AND INTERFACES

Task Requirement: Create abstract class shape with abstract method draw() implemented by circle and rectangle. Implement interface printable in class report.

7.1 UML Class Diagram

Abstraction & Interface Design



7.2 Java Source Code

```
package task1;

abstract class shape {
    abstract void draw();
}

class circle extends shape {
    @Override
    public void draw() {
        System.out.println("the drawing is circle");
    }
}

class rectangle extends shape {
    @Override
    public void draw() {
        System.out.println("the drawing is rectangle");
    }
}

interface printable {
    void ward();
}

class report implements printable {
    @Override
    public void ward() {
        System.out.println("the interface is impenents");
    }
}

public class Abstract {
    public static void main(String[] args) {
        report r = new report();
        r.ward();

        circle a = new circle();
        a.draw();

        rectangle t = new rectangle();
        t.draw();
    }
}
```

7.3 Code Explanation

Class shape defines a contractual abstract method draw() without execution details. Subclasses circle and rectangle fulfill this contract by overriding draw(). Interface printable specifies the method signature ward(), which is implemented in report with public access.

7.4 Execution Outputs

SUCCESS CASE Standard Abstract/Interface Execution:

```
the interface is impenents
the drawing is circle
the drawing is rectangle
```

```
shape s = new shape(); // Invalid instantiation
// Execution Result:
Abstract.java:31: error: shape is abstract; cannot be instantiated
shape s = new shape();
            ^
```

8. OBSERVATION QUESTIONS & ANSWERS

Q1. What is Object-Oriented Programming (OOP)?

Answer: Object-Oriented Programming is a programming paradigm based on the concept of "objects", which contain data (fields/attributes) and code (methods/functions). OOP aims to implement real-world entities such as inheritance, hiding, and polymorphism in programming to increase reusability, modularity, and maintainability.

Q2. Differentiate between a Class and an Object.

Answer:

Feature	Class	Object
Definition	A logical blueprint/template for creating objects.	A physical entity/instance of a class.
Memory Allocation	No memory is allocated when declared.	Allocates memory on the heap when created via new.
Existence	Declared once using the class keyword.	Can be instantiated multiple times.

Q3. Explain the purpose of constructors.

Answer: A constructor is a special block of code used to initialize objects when they are created. Key features include:

- It has the exact same name as the class and no explicit return type.
- It executes automatically when an object is instantiated via new.
- Default constructors provide default values; parameterized constructors allow custom initialization of instance variables.

Q4. What is Encapsulation? How is it implemented in Java?

Answer: Encapsulation is the mechanism of wrapping data (variables) and code acting on data (methods) together as a single unit, shielding fields from arbitrary external modification.

Implementation in Java:

1. Declare class variables/attributes as private.
2. Provide public getter (accessor) and setter (mutator) methods to read and modify private field values safely.

Q5. Differentiate between Method Overloading and Method Overriding.

Answer:

Property	Method Overloading	Method Overriding
Polymorphism Type	Compile-time (Static binding)	Run-time (Dynamic binding)
Location	Occurs within the same class.	Occurs across parent and child classes.
Signature	Must have different parameter lists.	Must have identical parameter lists and return type.

Q6. Explain the four pillars of Object-Oriented Programming with suitable examples.

Answer:

- **Encapsulation:** Bundling data and methods into a single unit (e.g., class books with private fields and public getters/setters).
- **Inheritance:** Mechanism where a new class acquires properties of an existing class (e.g., `Student` extends `Person`).
- **Polymorphism:** Ability of an object to take on many forms (e.g., `shape` calculation `area()` overloaded or `draw()` overridden).
- **Abstraction:** Hiding operational details and showing only functionality to the user (e.g., `abstract class shape` with `abstract draw()`).

Q7. Differentiate between Abstract Classes and Interfaces.

Answer:

Criteria	Abstract Class	Interface
Methods	Can have abstract and concrete methods.	Contains abstract methods (until Java 8 added default/static).
Inheritance	Subclasses use <code>extends</code> (Single inheritance).	Classes use <code>implements</code> (Multiple inheritance).
Variables	Can have final, non-final, static, and non-static variables.	Variables are implicitly <code>public static final</code> .